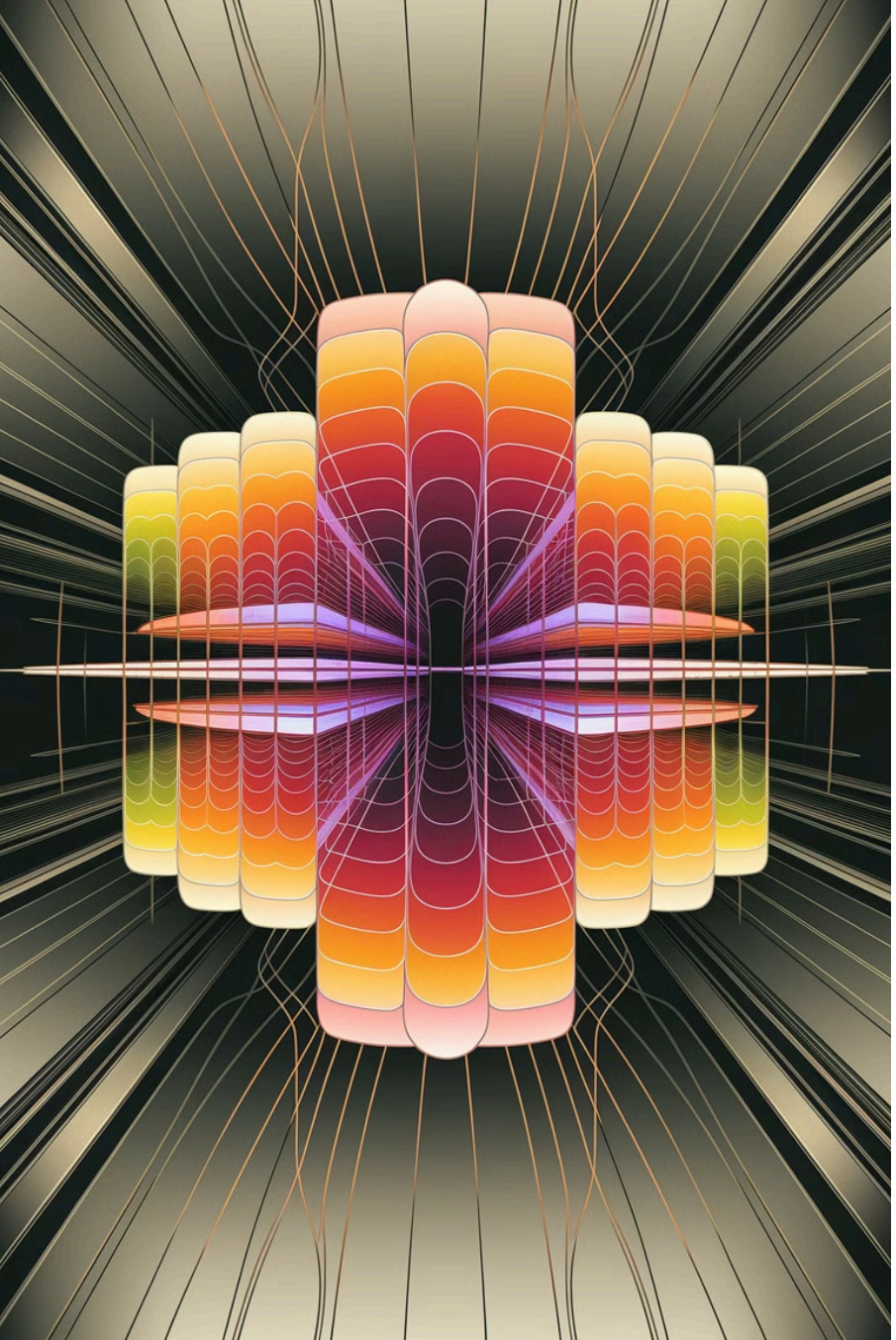


Lecture 6

Transformers for Language Modeling
Transformer Block and Architecture

MAI554: Deep Learning for NLP

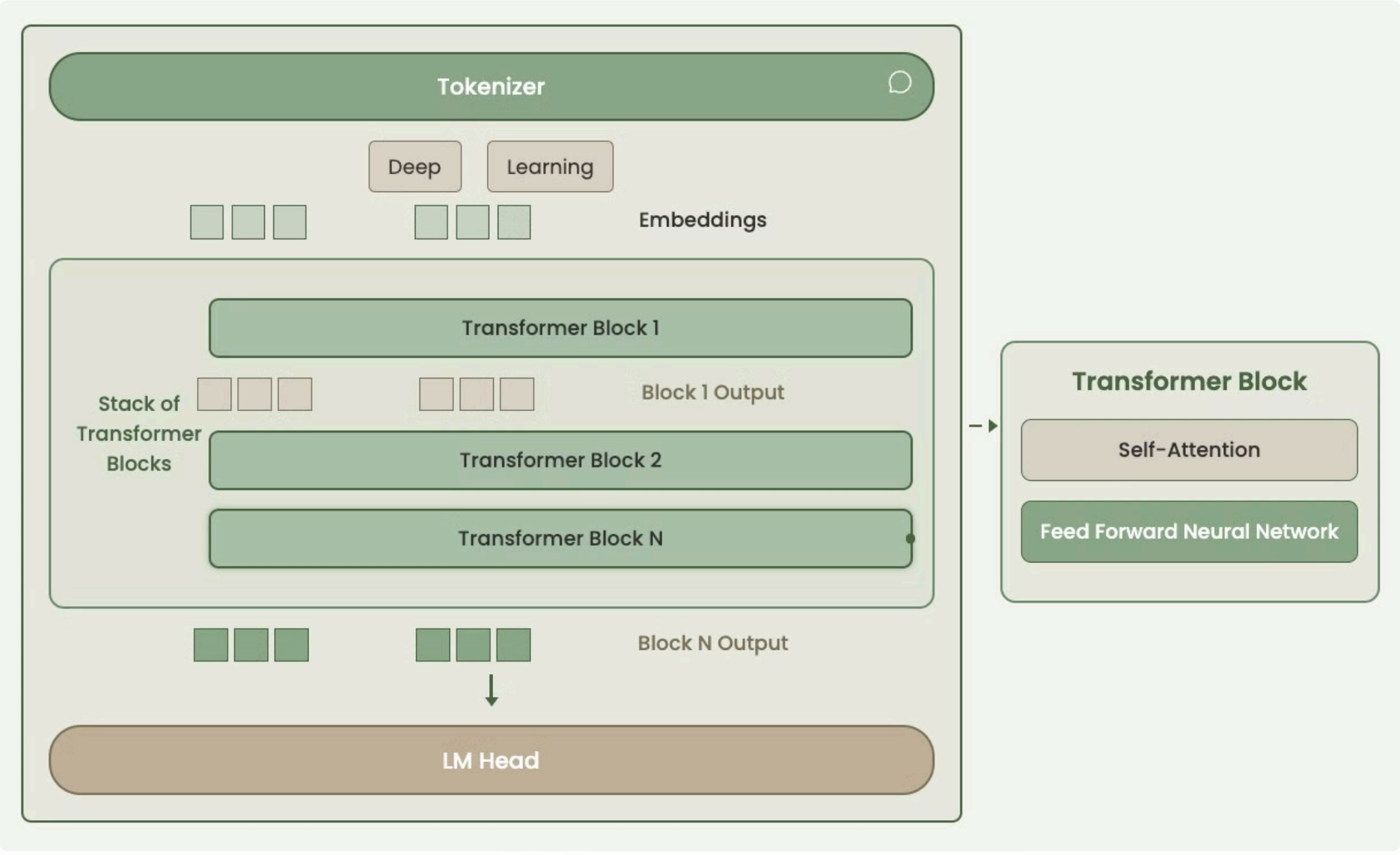
Prof. Anis Koubaa

Professor

College of Engineering
Alfaisal University

akoubaa@alfaisal.edu

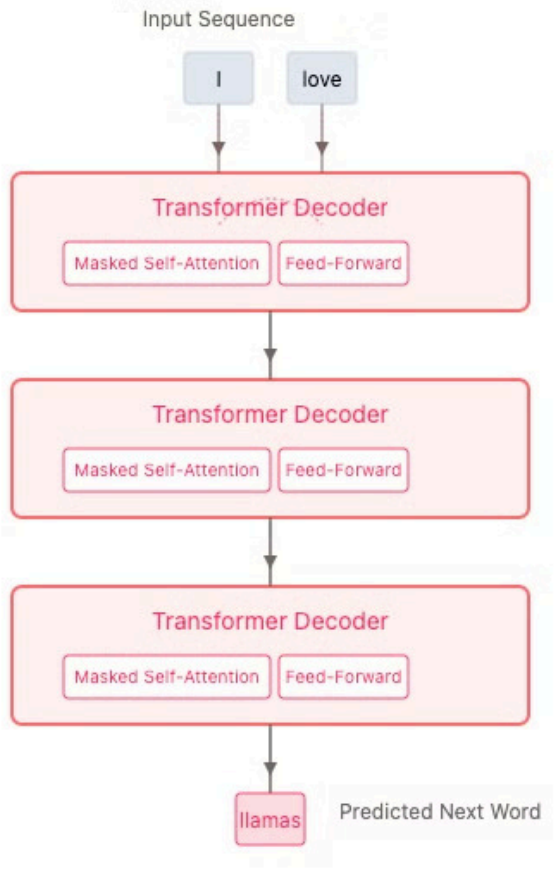
Transformer Block



Transformer Architecture Comparison

Generative Models (GPT-1)

Decoder-Only Architecture

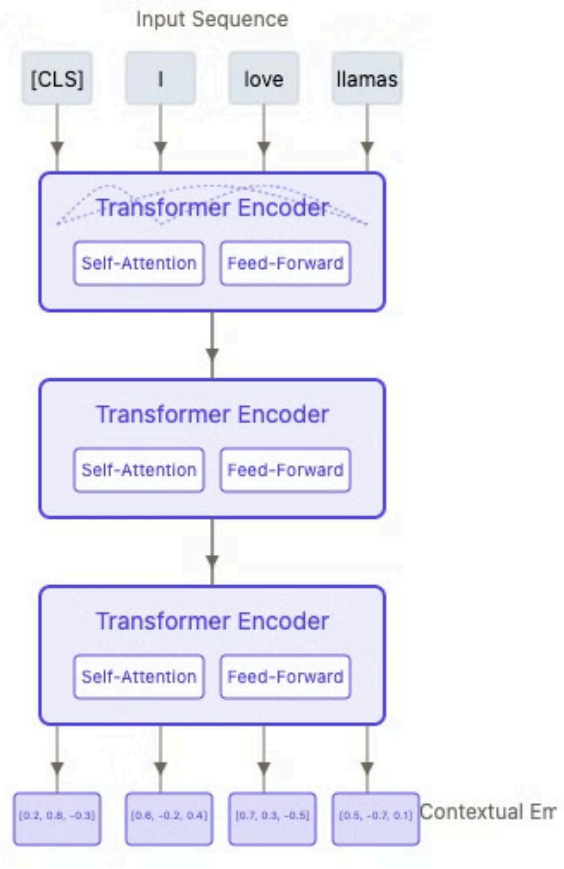


- **Decoder-Only Architecture** - Uses only transformer decoder layers
- **Masked Self-Attention** - Can only see previous tokens, not future ones
- **Autoregressive** - Outputs one token at a time, using previous outputs as inputs
- **Left-to-Right Processing** - Processes text in a single direction

Example Tasks: *Text generation, completion, summarization, and translation*

Representation Models (BERT)

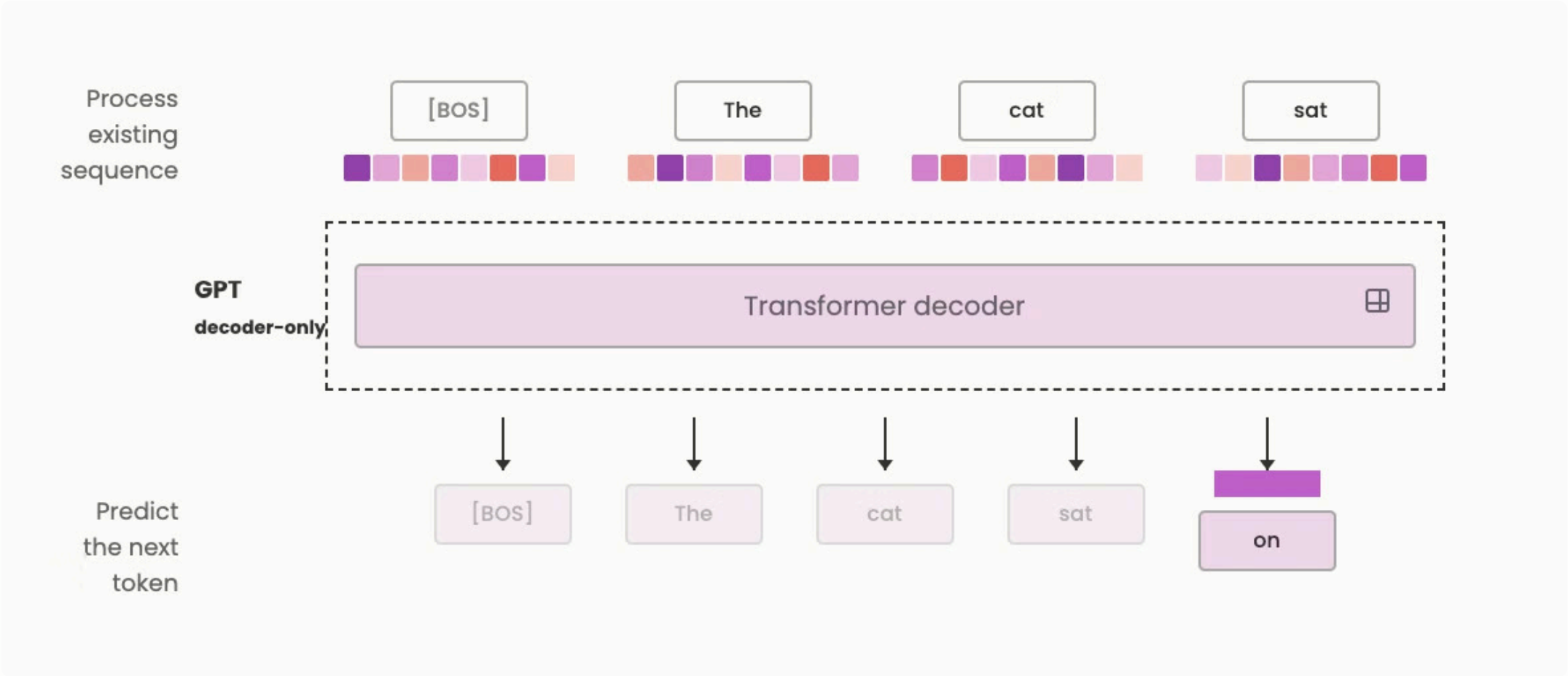
Encoder-Only Architecture



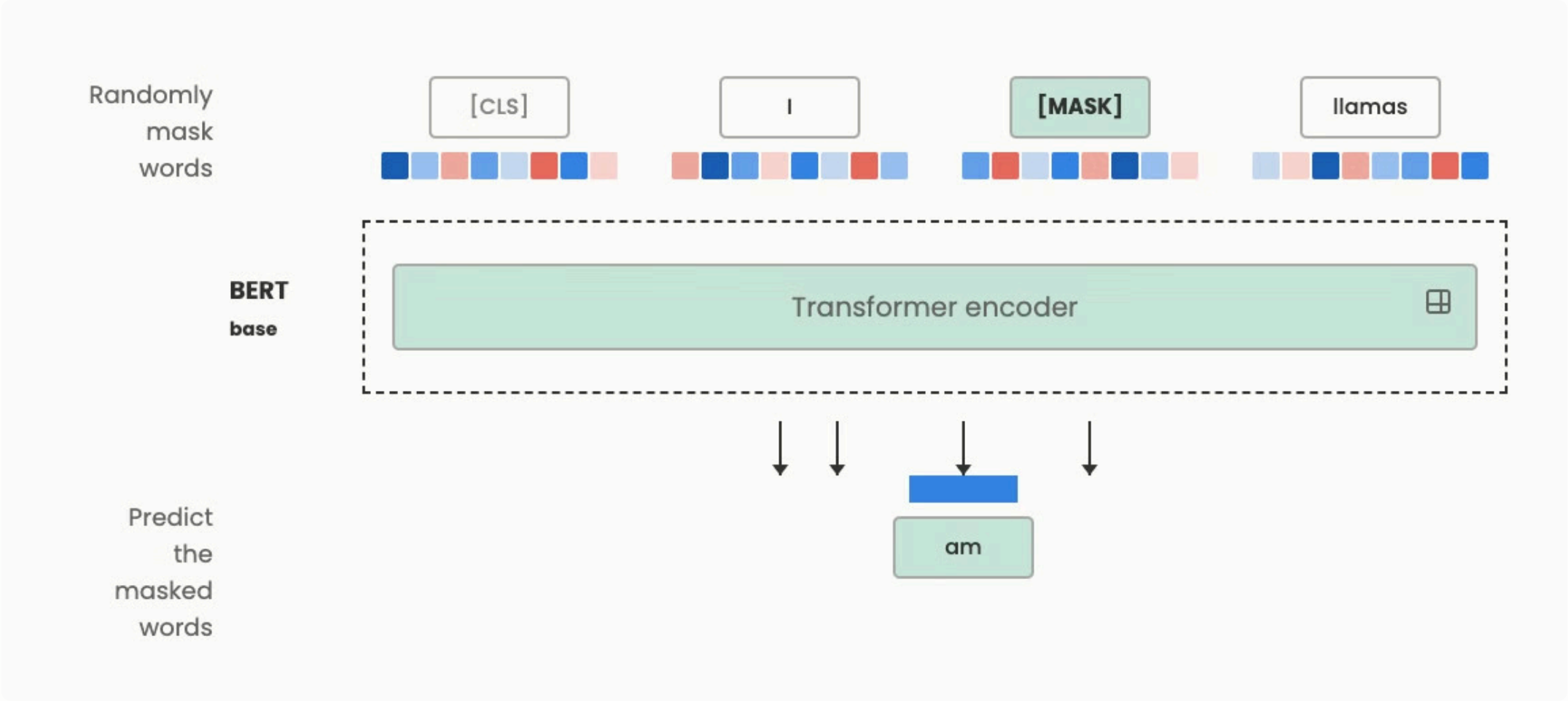
- **Encoder-Only Architecture** - Uses only transformer encoder layers
- **Bidirectional Self-Attention** - Can see all tokens in the sequence simultaneously
- **Contextual Representations** - Outputs embeddings for all tokens in the sequence
- **Special Tokens** - Uses [CLS] token for classification tasks and [SEP] for sequence separation

Example Tasks: *Sentence classification, named entity recognition, question answering, and token classification*

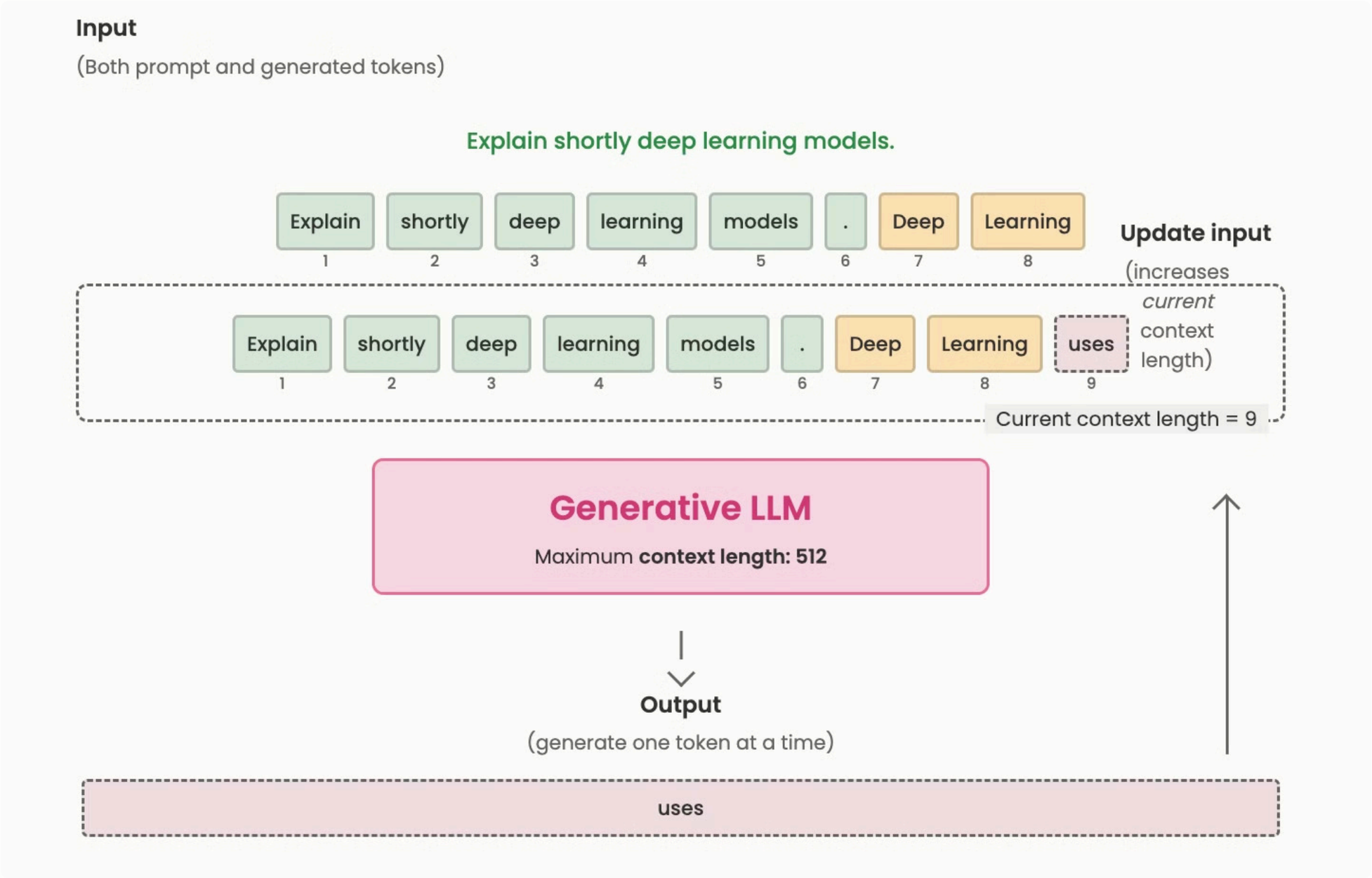
Generative Models



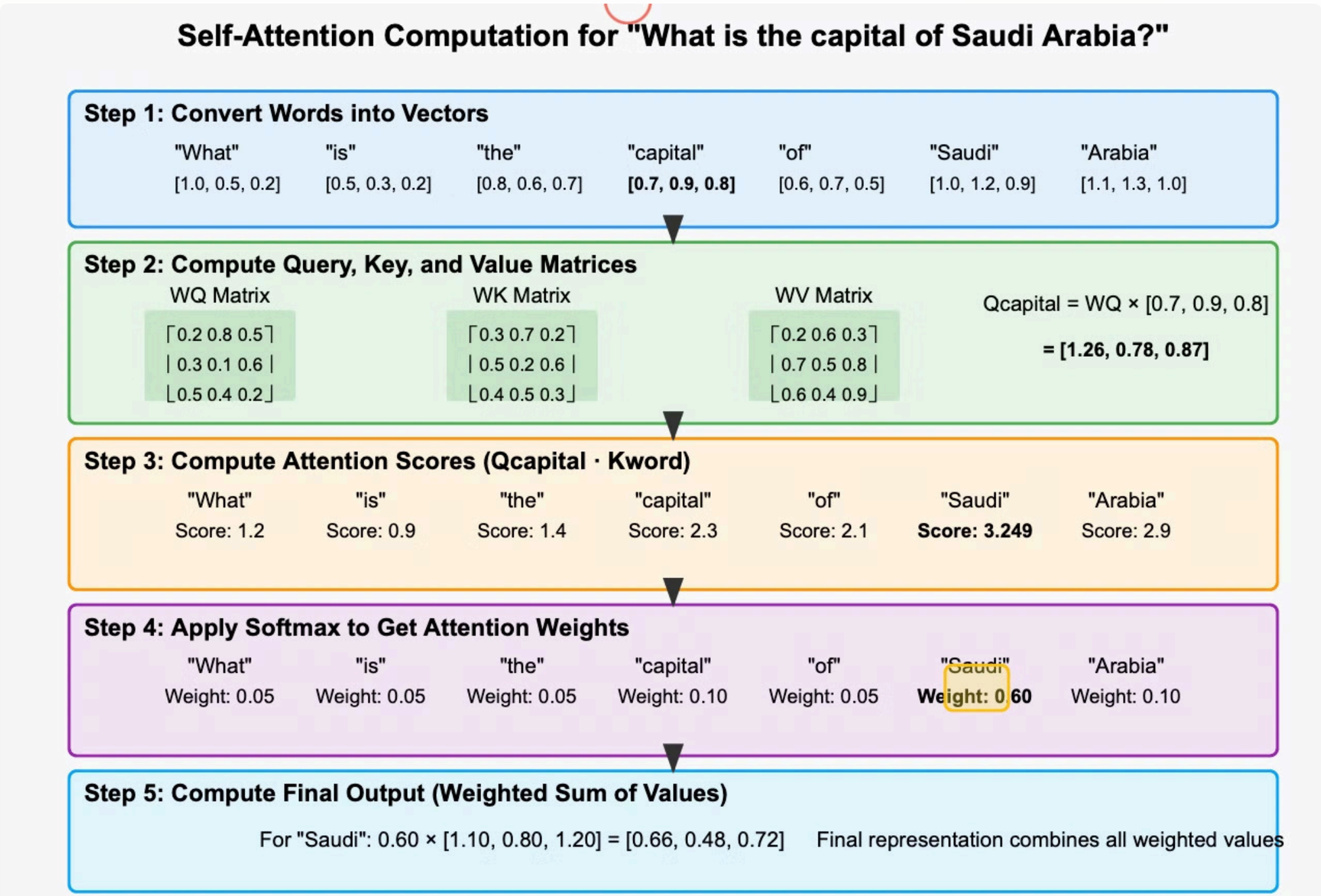
Representational Models



Token Generation Process



Step by Step Calculations



Self-Attention Head

Self-Attention Head

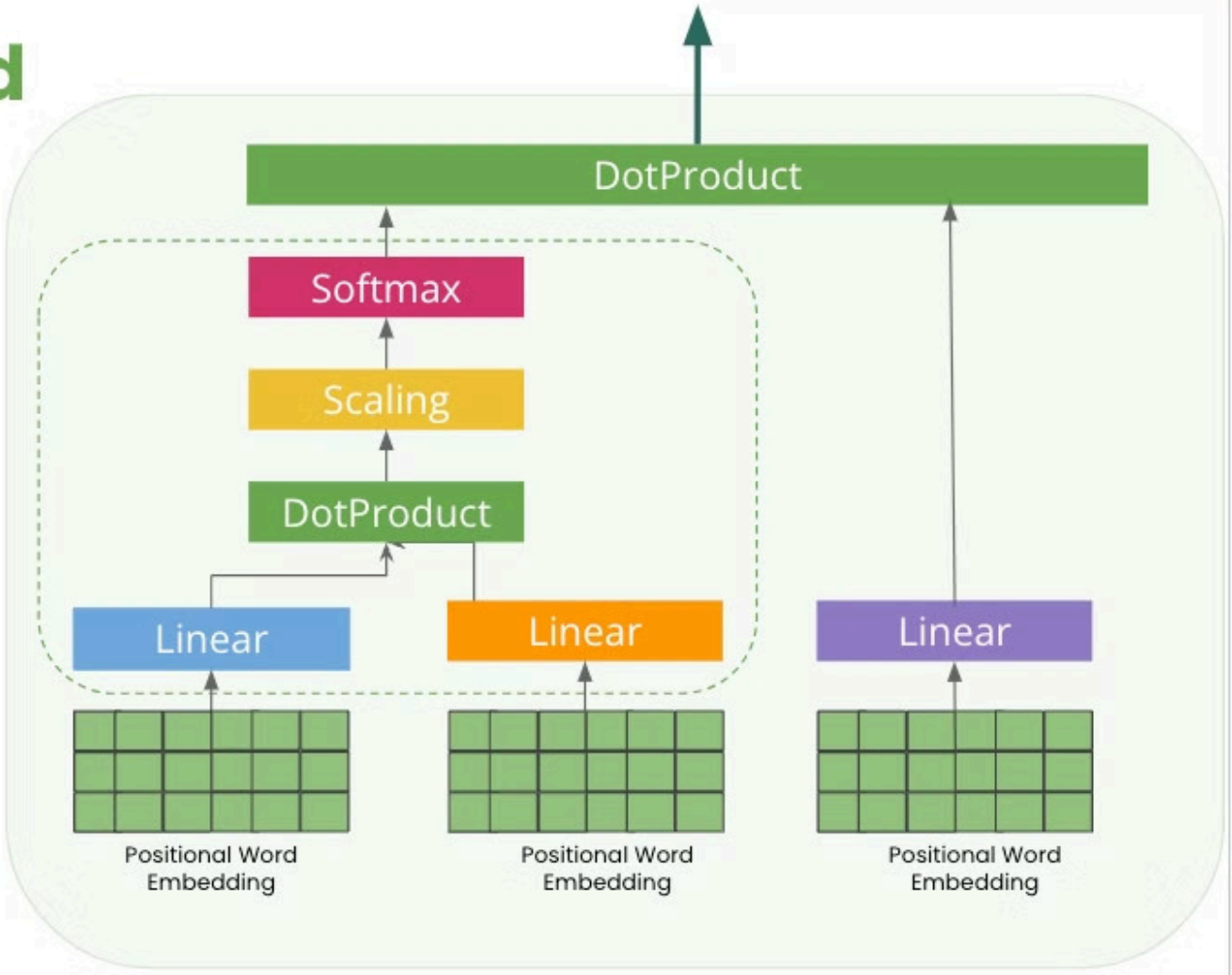
```python
import torch
from torch import nn

def scaled_dot_product_attention(query: Tensor, key: Tensor, value: Tensor) -> Tensor:
    score = query.bmm(key.transpose(1, 2))
    scale = key.size(2) ** 0.5 #equivalent to scale = key.size(-1) ** 0.5
    scaled_score = score / scale
    softmax_score = funct.softmax(scaled_score, dim=-1)
    return softmax_score.bmm(value)

class AttentionHead(nn.Module):
    def __init__(self, dim_in: int, dim_q: int, dim_k: int):
        super().__init__()
        self.q = nn.Linear(dim_in, dim_q)
        self.k = nn.Linear(dim_in, dim_k)
        self.v = nn.Linear(dim_in, dim_k)

    def forward(self, query: Tensor, key: Tensor, value: Tensor) -> Tensor:
        return scaled_dot_product_attention(self.q(query), self.k(key), self.v(value))
```

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Self-Attention Head

selfattention.py
~/Desktop/selfattention.py

```python
from torch import nn

def scaled_masked_dot_product_attention(query: Tensor, key: Tensor, value: Tensor, mask: Tensor) -> Tensor:
    score = query.bmm(key.transpose(1, 2))
    scale = key.size(2) ** 0.5
    scaled_score = score / scale

    if mask is not None:
        scaled_score = scaled_score.masked_fill(mask == 0, -1e9)

    softmax_score = torch.nn.functional.softmax(scaled_score, dim=-1)
    return softmax_score.bmm(value)


class AttentionHead(nn.Module):
    def __init__(self, dim_in: int, dim_q: int, dim_k: int, mask: Tensor = None):
        super().__init__()
        self.q = nn.Linear(dim_in, dim_q)
        self.k = nn.Linear(dim_in, dim_k)
        self.v = nn.Linear(dim_in, dim_k)

    def forward(self, query: torch.Tensor, key: torch.Tensor, value: torch.Tensor, mask: torch.Tensor = None)
        return scaled_dot_product_attention(self.q(query), self.k(key), self.v(value), mask)
```



Masked Self-Attention

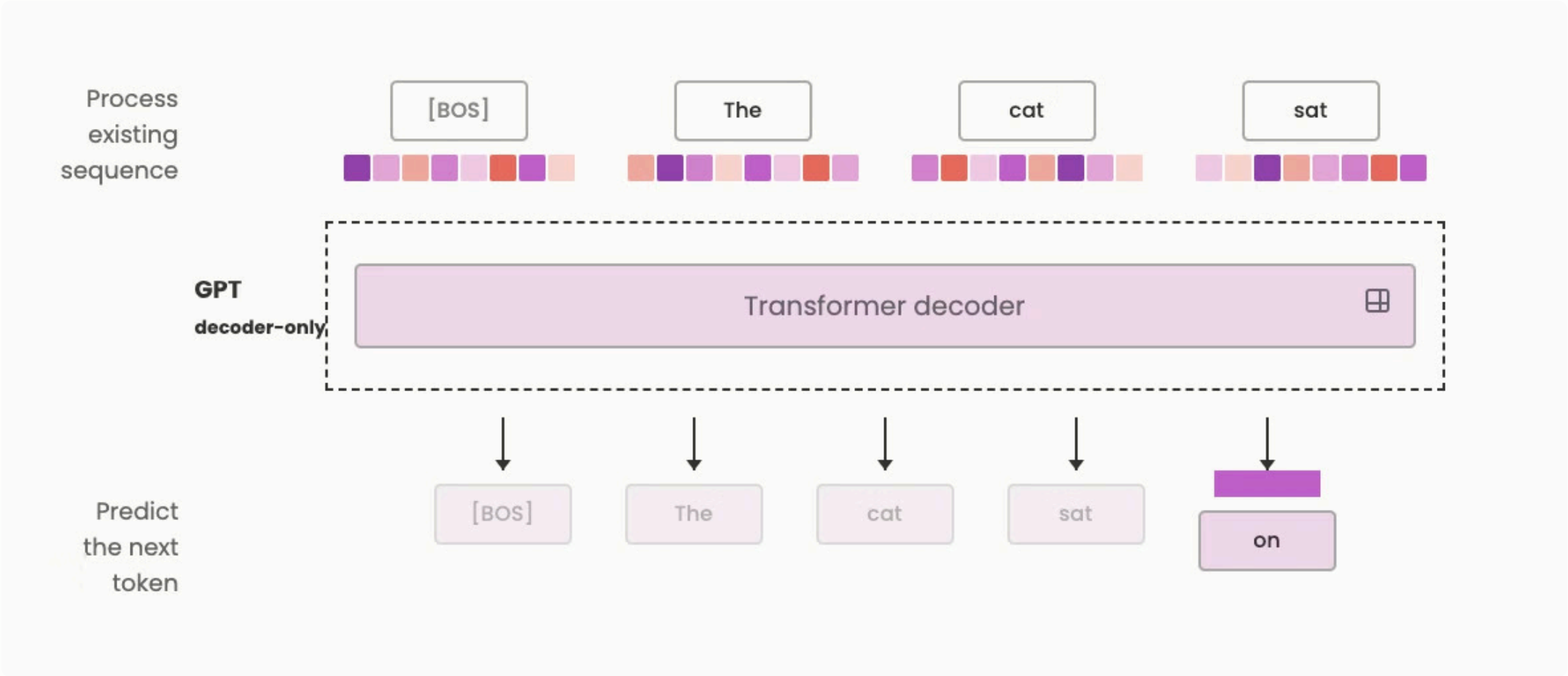
Causal Masks

Prevent "cheating" in autoregressive tasks (GPT)

Padding Masks

Used in encoder-only tasks

Generative Models



Masked Self-Attention in Transformers

Causal Masks (Future Masking)

Why Do We Need Causal Masks?

- In autoregressive models (like GPT), tokens are generated sequentially
- The model must not see future tokens when generating
- Without causal masks, the model would "cheat" by using information it shouldn't have access to yet

How Does Causal Masking Work?

The mask prevents each token from attending to future tokens by setting those attention scores to -∞. After softmax, these become 0, effectively blocking information flow from future tokens.

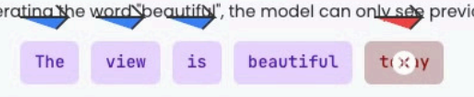
Causal Mask for a 5-token sequence

Token	t_1	t_2	t_3	t_4	t_5
t_1	0	-∞	-∞	-∞	-∞
t_2	0	0	-∞	-∞	-∞
t_3	0	0	0	-∞	-∞
t_4	0	0	0	0	-∞
t_5	0	0	0	0	0

$$M_{ij} = \{ 0, \text{if } j \le i \text{ (allow past \& present tokens)}\ -\infty, \text{if } j > i \text{ (mask future tokens)} \}$$

Causal Mask in Action

When generating the word "beautiful", the model can only see previous words



Where Are Causal Masks Used?

GPT · LLaMA · Decoder-only Transformers · Machine Translation (Decoder)

Padding Masks

Why Do We Need Padding Masks?

- Sentences in natural language have different lengths
- Transformers process sentences in batches, requiring padding to maintain fixed dimensions
- Without padding masks, the model would attend to meaningless padding tokens

How Does Padding Masking Work?

Padding masks ensure that real tokens cannot attend to padding tokens by setting those attention scores to -∞. After softmax, these become 0, effectively preventing information flow from padding tokens.

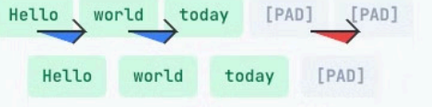
Padding Mask for a batch with variable-length sequences

Token	t_1	t_2	t_3	PAD	PAD
t_1	0	0	0	-∞	-∞
t_2	0	0	0	-∞	-∞
t_3	0	0	0	-∞	-∞
PAD	-∞	-∞	-∞	-∞	-∞
PAD	-∞	-∞	-∞	-∞	-∞

$$M_{ij} = \{ 0, \text{if } j \text{ is a real token}\ -\infty, \text{if } j \text{ is a padding token} \}$$

Padding Mask in Action

Processing a shorter sentence with padding in a batch



Where Are Padding Masks Used?

BERT · RoBERTa · Encoder-only Transformers · Machine Translation (Encoder)

Masked Self-Attention

selfattention.py
~/Desktop/selfattention.py

```python
from torch import nn

def scaled_masked_dot_product_attention(query: Tensor, key: Tensor, value: Tensor, mask: Tensor) -> Tensor:
    score = query.bmm(key.transpose(1, 2))
    scale = key.size(2) ** 0.5
    scaled_score = score / scale

    if mask is not None:
        scaled_score = scaled_score.masked_fill(mask == 0, -1e9)

    softmax_score = torch.nn.functional.softmax(scaled_score, dim=-1)
    return softmax_score.bmm(value)


class AttentionHead(nn.Module):
    def __init__(self, dim_in: int, dim_q: int, dim_k: int, mask: Tensor = None):
        super().__init__()
        self.q = nn.Linear(dim_in, dim_q)
        self.k = nn.Linear(dim_in, dim_k)
        self.v = nn.Linear(dim_in, dim_k)

    def forward(self, query: torch.Tensor, key: torch.Tensor, value: torch.Tensor, mask: torch.Tensor = None)
        return scaled_dot_product_attention(self.q(query), self.k(key), self.v(value), mask)

```

Masked Self-Attention in Transformers

Causal Masks (Future Masking)

Why Do We Need Causal Masks?

- In autoregressive models (like GPT), tokens are generated sequentially
- The model must not see future tokens when generating
- Without causal masks, the model would "cheat" by using information it shouldn't have access to yet

How Does Causal Masking Work?

The mask prevents each token from attending to future tokens by setting those attention scores to -∞. After softmax, these become 0, effectively blocking information flow from future tokens.

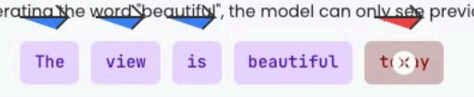
Causal Mask for a 5-token sequence

Token	t_1	t_2	t_3	t_4	t_5
t_1	0	-∞	-∞	-∞	-∞
t_2	0	0	-∞	-∞	-∞
t_3	0	0	0	-∞	-∞
t_4	0	0	0	0	-∞
t_5	0	0	0	0	0

$$M_{ij} = \{ 0, \text{if } j \le i \text{ (allow past \& present tokens)}\ -\infty, \text{if } j > i \text{ (mask future tokens)} \}$$

Causal Mask in Action

When generating the word "beautiful", the model can only see previous words



Where Are Causal Masks Used?

GPT · LLaMA · Decoder-only Transformers · Machine Translation (Decoder)

Padding Masks

Why Do We Need Padding Masks?

- Sentences in natural language have different lengths
- Transformers process sentences in batches, requiring padding to maintain fixed dimensions
- Without padding masks, the model would attend to meaningless padding tokens

How Does Padding Masking Work?

Padding masks ensure that real tokens cannot attend to padding tokens by setting those attention scores to -∞. After softmax, these become 0, effectively preventing information flow from padding tokens.

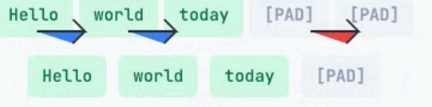
Padding Mask for a batch with variable-length sequences

Token	t_1	t_2	t_3	PAD	PAD
t_1	0	0	0	-∞	-∞
t_2	0	0	0	-∞	-∞
t_3	0	0	0	-∞	-∞
PAD	-∞	-∞	-∞	-∞	-∞
PAD	-∞	-∞	-∞	-∞	-∞

$$M_{ij} = \{ 0, \text{if } j \text{ is a real token}\ -\infty, \text{if } j \text{ is a padding token} \}$$

Padding Mask in Action

Processing a shorter sentence with padding in a batch



Where Are Padding Masks Used?

BERT · RoBERTa · Encoder-only Transformers · Machine Translation (Encoder)



Multihead Self-Attention

Parallel Processing

Multiple attention heads learn different aspects of relationships in parallel

Improved Representation

Combines outputs from different heads to form richer representations

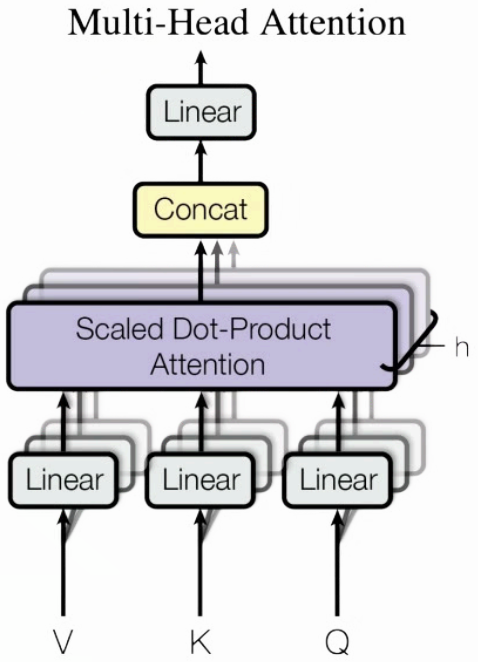
Multi-Head Attention - Motivation

Why Multiple Heads?

- **Self-attention** is **powerful** because it lets each **token** in a sequence **focus** on other tokens **dynamically**.
- **But** a **single self-attention** operation might focus too **narrowly** on **one** type of **relationship** (e.g., **nearby tokens** or specific **semantic links**).

Key Concept

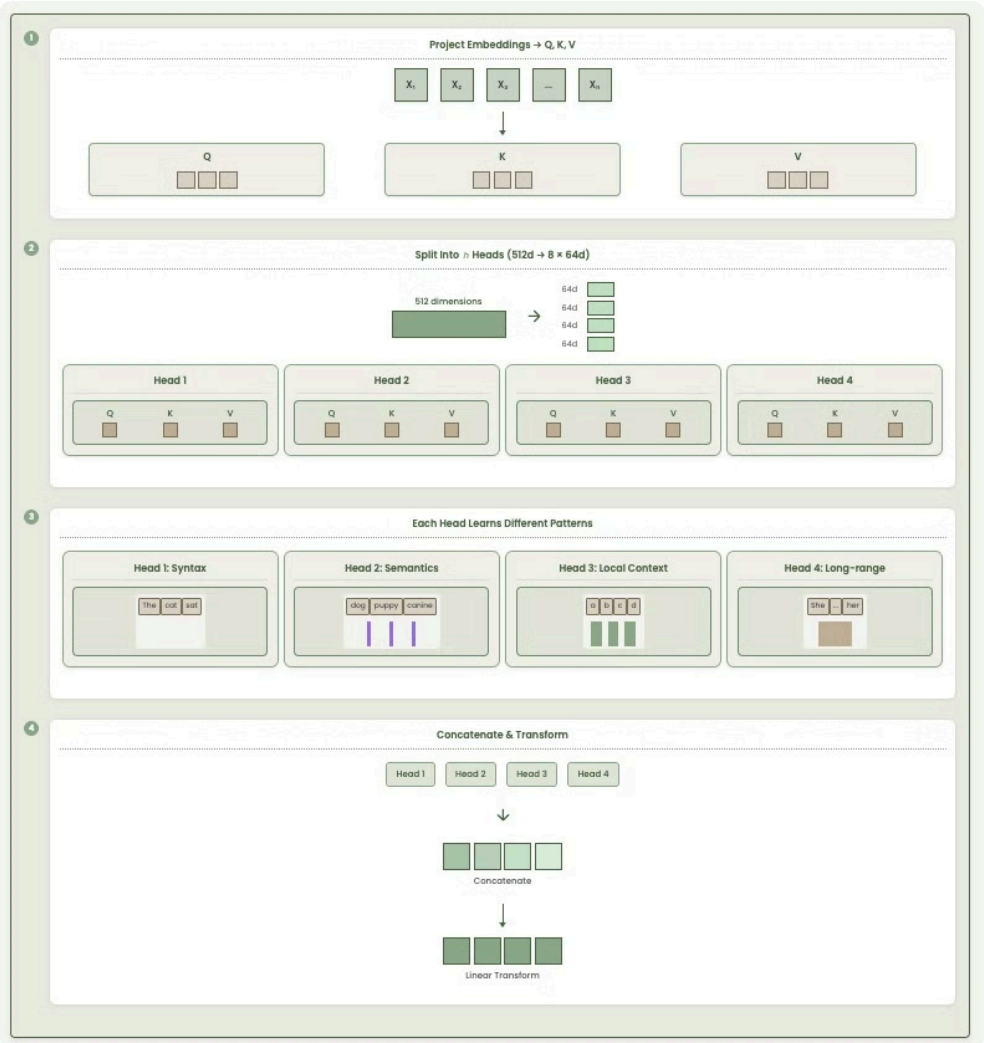
- **Multi-head attention** runs multiple **self-attention** operations in **parallel**,
- Allow the model to capture different types of **relationships** simultaneously.



Multi-Head Attention - How it Works

How It Works

- **Multiple Projections:** Input **embeddings** are projected into multiple sets of **Query**, **Key**, and **Value** vectors
  - Creates different representation spaces for the same input data
- **Dimension Splitting:** Each **"head"** operates on a **subset** of the embedding dimensions (e.g., 8 heads with 64 dimensions each for a 512d model)
  - Allows for more efficient computation while maintaining representation power
- **Parallel Learning:** Heads **independently** learn different patterns - some might focus on **syntax**, others on **semantic relationships**
  - Helps capture various aspects of the input simultaneously
- **Output Integration:** Outputs from all heads are **concatenated** and transformed to combine insights
  - Final linear projection merges the different attention patterns into a cohesive representation

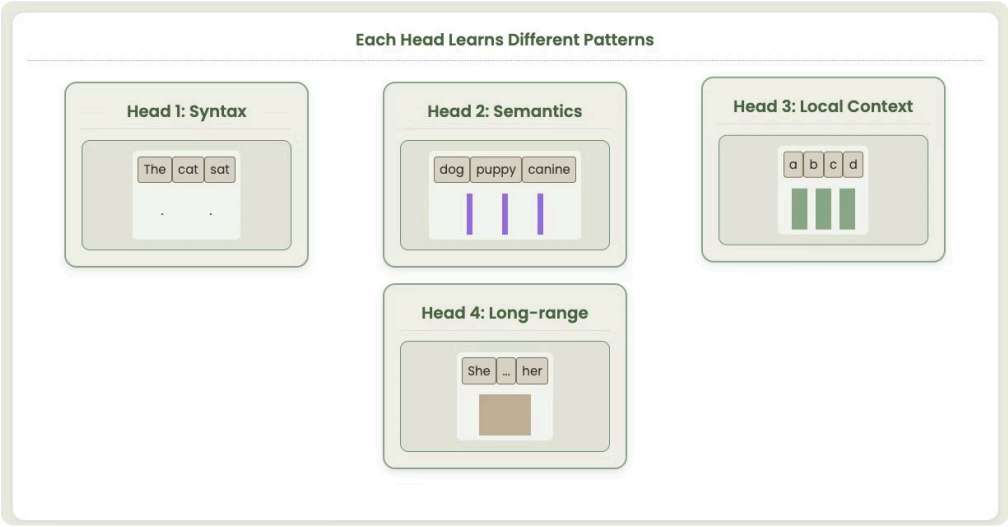


Multi-Head Attention - Why It's Effective

Specialized Attention Patterns

Each attention head becomes a specialized **feature extractor (expert)** focusing on different **linguistic patterns**.

- Some heads capture **local context** between adjacent tokens
- Others track **long-distance dependencies** across sentences
- Many develop sensitivity to specific **syntactic relationships**



Intuition

Think of it like having **multiple experts analyzing the same text** from different angles, then pooling their findings for a richer understanding.

Components of the Transformer Block - Representation Models

Input Embeddings

Token embeddings enter the block, often with positional encodings already added.

Multi-Head Attention

Tokens exchange information across the sequence through parallel attention mechanisms.

Add & Norm

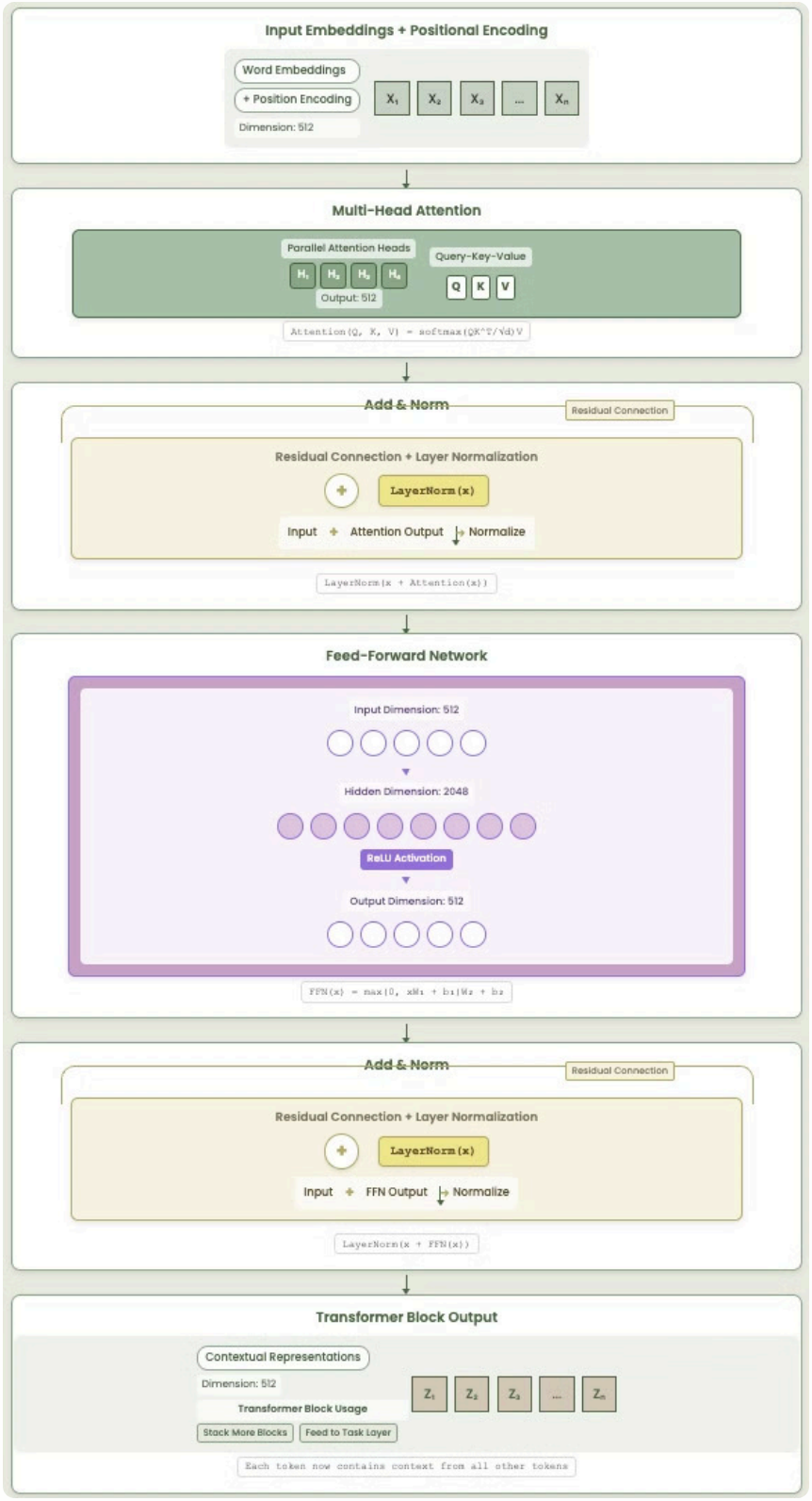
Residual connection adds original input to attention output. Layer normalization stabilizes training.

Feed-Forward Network

Each token processes its updated representation independently through a small neural network.

Final Add & Norm

Another residual connection and normalization layer complete the transformer block.



Components of the Transformer Block Generative Models

Input Embeddings

Token embeddings enter the block, often with positional encodings already added.

Multi-Head Attention

Tokens exchange information across the sequence through parallel attention mechanisms.

Add & Norm

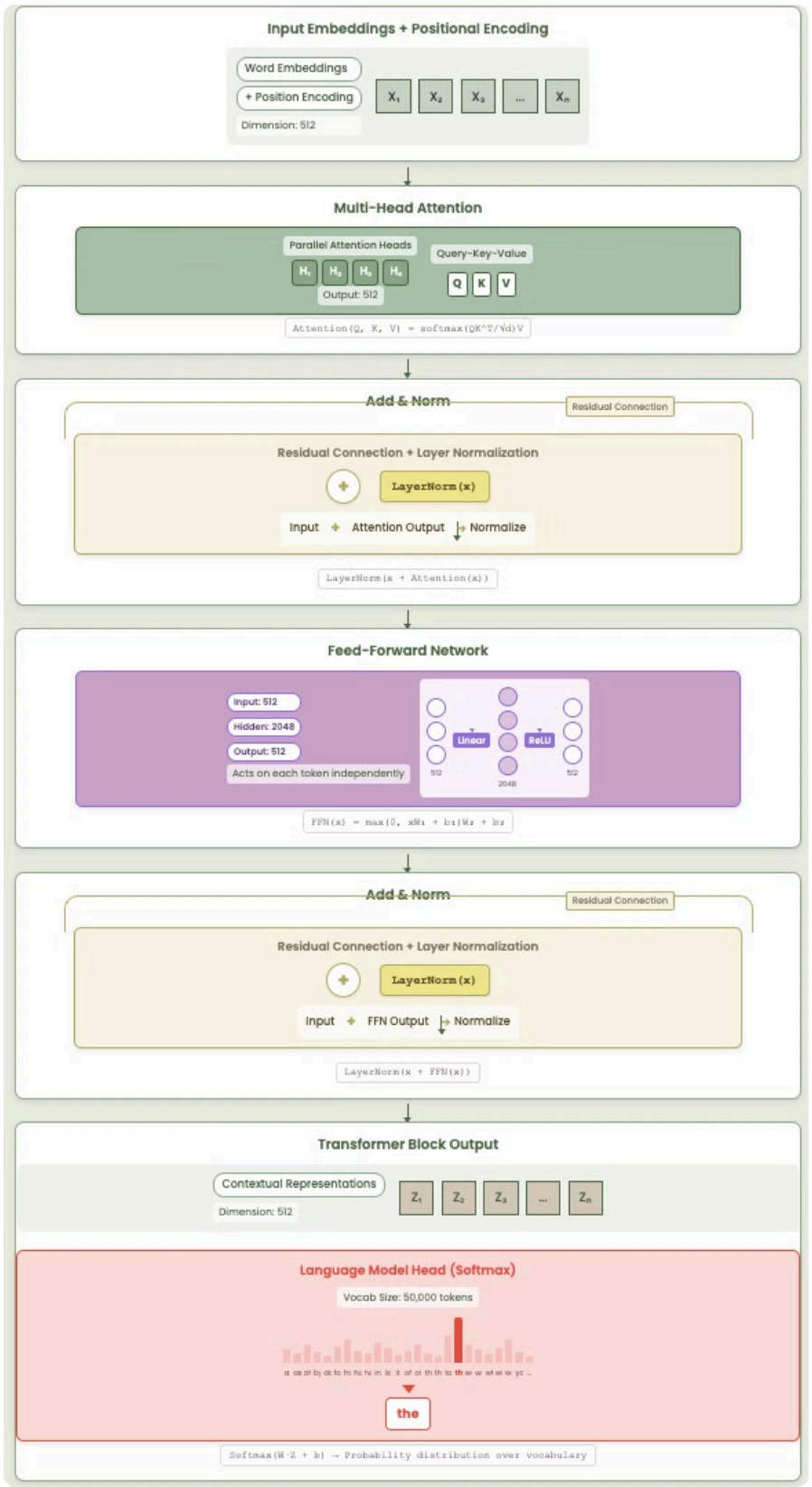
Residual connection adds original input to attention output. Layer normalization stabilizes training.

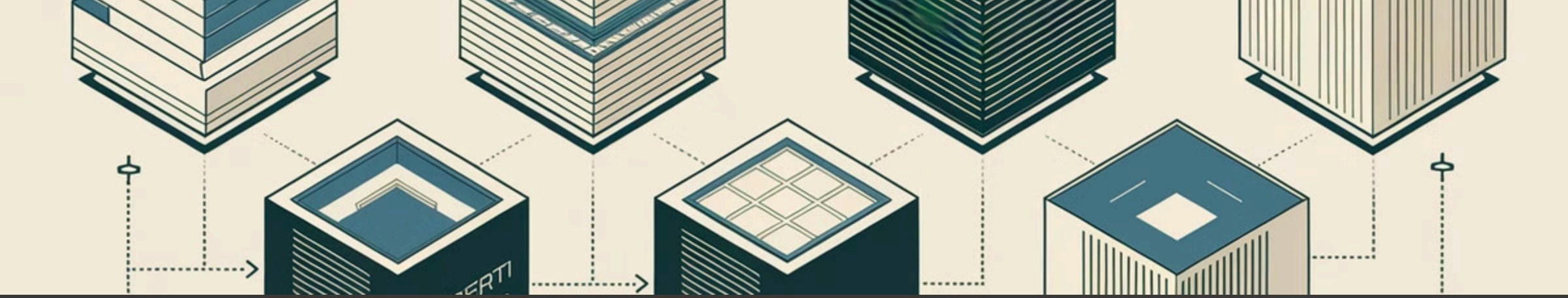
Feed-Forward Network

Each token processes its updated representation independently through a small neural network.

Final Add & Norm

Another residual connection and normalization layer complete the transformer block.





Mixture of Experts (MoE)

Specialized Computation

Multiple expert networks that specialize in different aspects of input processing

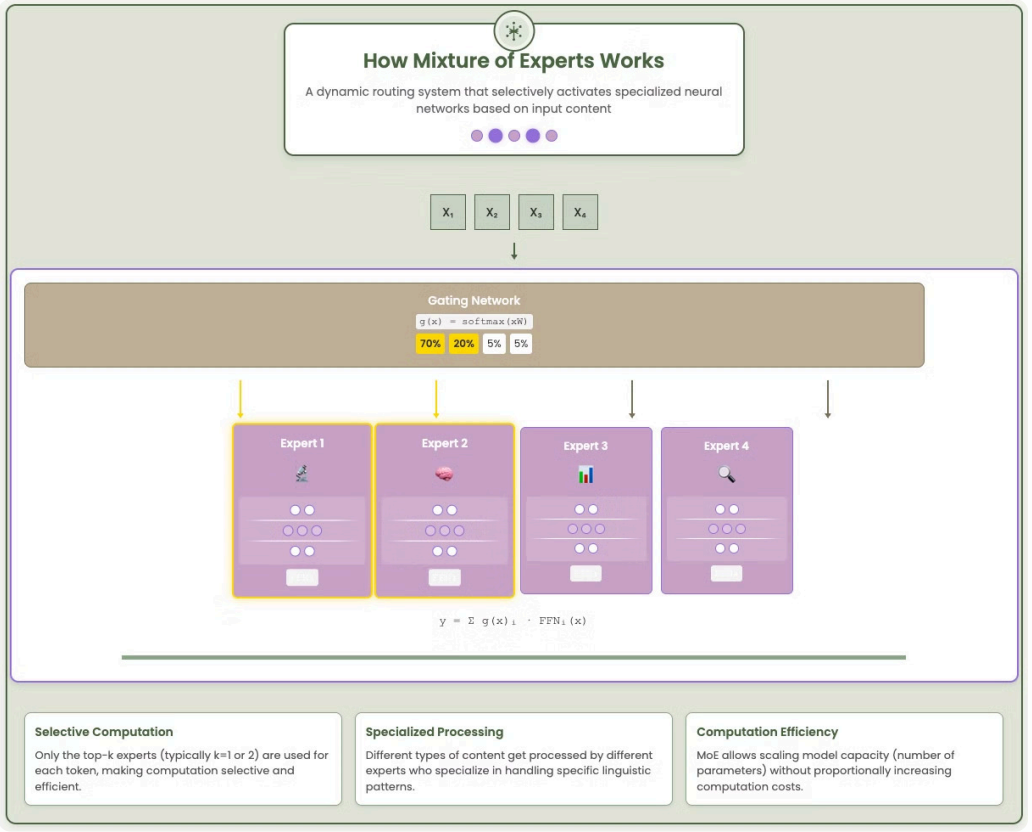
Conditional Activation

Router network dynamically selects which experts to use for each input, improving efficiency

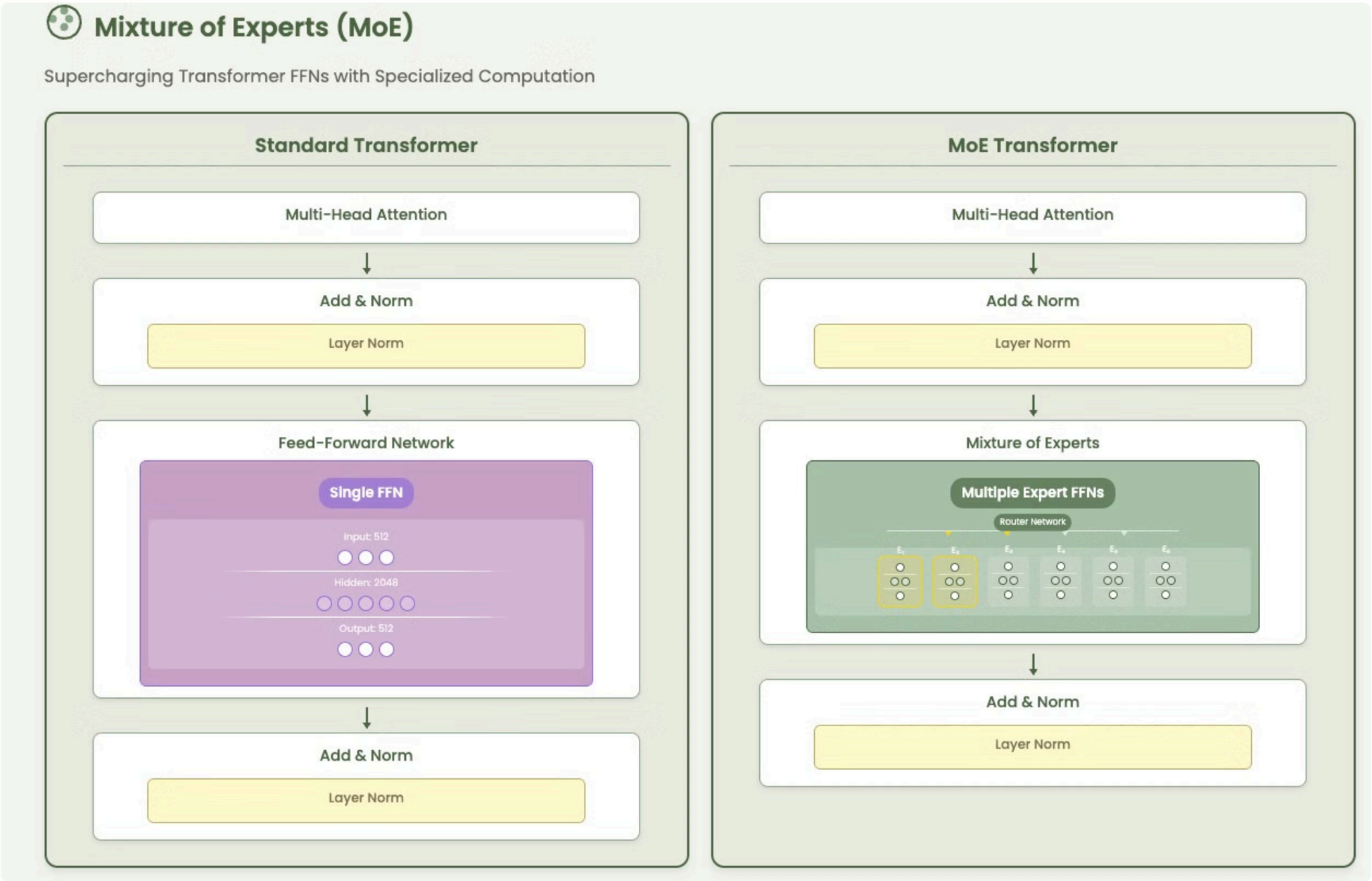
Mixture of Experts (MoE)

MoE **supercharges** the Transformer's **feed-forward network** (FFN) layer while maintaining **computational efficiency**.

1. **Motivation**:
    - **Standard Transformers**: Every token processes through the same FFN—*inefficient* for **billion-parameter models** where tokens have different needs.
    - **MoE solution**: Assign **specialized FFNs** to tokens based on content requirements, enabling **selective computation**.
2. **Core Concept**:
    - Replace the single FFN with a ***Mixture of Experts*** layer.
    - **Components**:
        - ***n** expert networks* (specialized FFNs)
        - A ***gating network*** that **routes tokens** to appropriate experts



Standard Transformer vs. Mixture of Experts



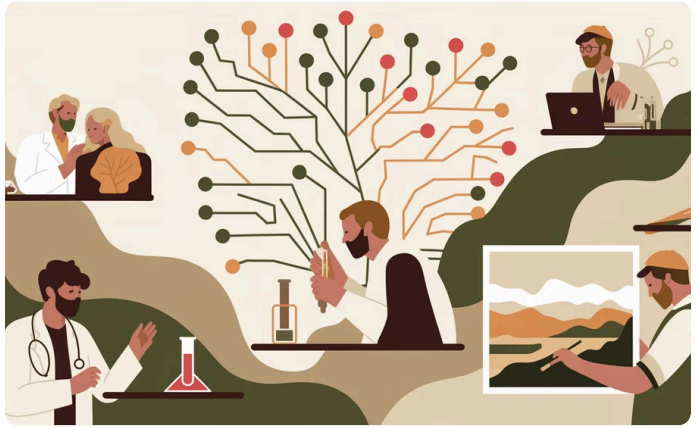
Why MoE Is Powerful



Scalability

Models can include **thousands of experts** while activating only a **few per token**.

This creates **massive parameter capacity** without proportional **computation costs**.



Specialization

Experts develop **distinct skills** across **syntax**, **vocabulary**, and **domain knowledge**.

Like specialized doctors, each expert handles **specific linguistic patterns**.

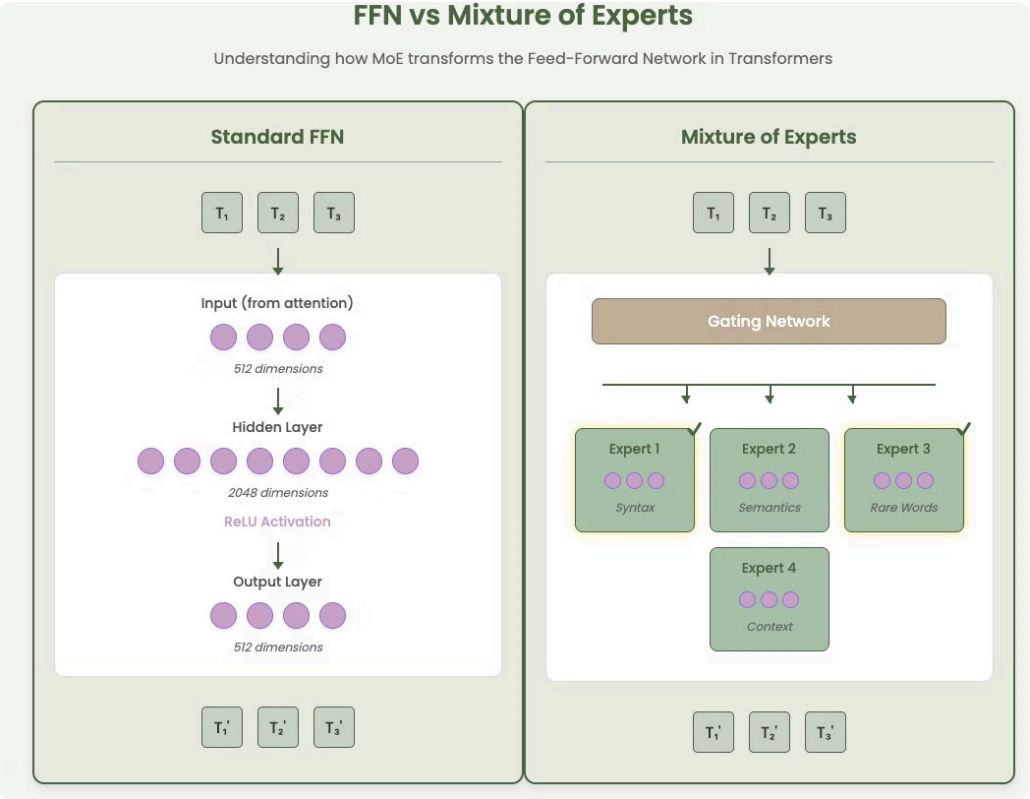


Efficiency

MoE models achieve **superior performance** with **less computation** than dense models.

The **router network** acts like triage, directing tokens only to **relevant experts**.

FFN vs Mixture of Experts



FFN Structure & Purpose

The Feed-Forward Network in a Transformer is a two-layer neural network that processes each token independently:

- First layer expands dimensionality (512→2048)
- ReLU activation adds non-linearity
- Second layer projects back to original size (2048→512)

It allows each token to "digest" its contextualized information after attention and refine it into a more useful representation.

MoE Structure & Purpose

Mixture of Experts replaces the single FFN with multiple specialized FFNs (experts):

- Gating network determines which experts to use for each token
- Only top-k experts (typically k=1 or 2) process each token
- Each expert can specialize in different linguistic patterns
- Final output combines contributions from selected experts

This approach increases model capacity without proportionally increasing computation.

Key Differences

- **Universality vs. Specialization:** FFN applies the same transformation to all tokens; MoE routes tokens to specialized experts.
- **Computation:** FFN activates all parameters for every token; MoE activates only a subset of parameters.
- **Scaling:** MoE can scale model capacity (parameters) more efficiently than standard FFN.
- **Adaptability:** MoE can adapt to different types of linguistic patterns by developing specialized experts.

Shared Goals

Both the FFN and MoE serve the same fundamental purpose in the Transformer architecture:

- Transform token representations after attention processing
- Apply non-linear transformations to capture complex patterns
- Provide each token a chance to refine its representation independently
- Prepare tokens for the next layer or final output

Restaurant Analogy

👨‍🍳 Standard FFN

Like a single, multi-talented chef cooking every dish on the menu for every customer. The chef must be skilled at everything but may not specialize deeply in any one cuisine.

🍽️ Mixture of Experts

Like a restaurant with multiple specialized chefs (Italian, Japanese, French, etc.), where a host (gating network) assigns each customer to the chef best suited for their order. You get tailored results with less wasted effort.